

ASSESSMENT TOOL-1

Compiler. Design.

Name :- V. Perarajan.

Reg. NO :- 192521404

Course Code :- CSA1405.

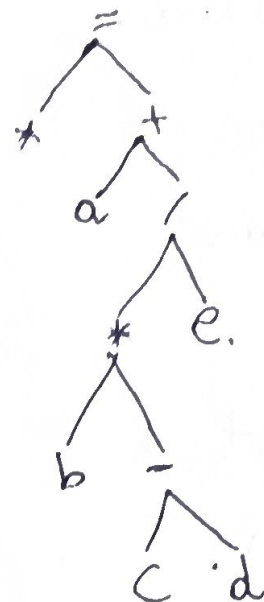
1. Compiler Phases for $x = a + b * (c - d) / e$.

a) Lexical Analysis (Tokens).

<u>Lexeme</u>	<u>Token.</u>
x	Identifier.
=	Assignment operator.
a	Identifier.
+	Plus operator.
b	Identifier.
*	Multiplication operator.
(	Left parenthesis.
c	Identifier.
-	Minus operator.
d	Identifier.
)	Right parenthesis.
/	division operator.
e	Identifier.

Total Tokens = 13.

b) Syntax Tree:-



(c) Intermediate code. (Three-Address code).

$$t_1 = c - d.$$

$$t_2 = b * t_1$$

$$t_3 = t_2 / e$$

$$t_4 = a + t_3$$

$$x = t_4$$

d) Optimization:-

Remove unnecessary temporary variables (if possible).

Constant folding (not applicable here).

Dead code elimination (not applicable).

e) Target Code:- (EX):-

LOADC, SUBd, MULb, DIVE, ADDa, STOREx.

2. Tokenization of `int value = count + + + - - num ;`

Tokens:-

<u>Lexeme</u>	<u>Token</u>
int	Keyword.
value	Identifier.
=	Assignment.
count	Identifier.
++	Increment operator.
+	plus operator.
--	Decrement operator.
num	Identifier.
;	Separator.

Explanation:-

⇒ The lexer follows the longest match.

rule:-

⇒ `count + + + - - num`.

⇒ `count + + + - - num`

not

⇒ `count + + + - - num`.

Updated LE x Rule:-

"++" {return INC;}

--" {return DEC;}

"+" {return PLUS;}

"-" {return MINUS;}

and Justification:-

Longest - match endured ++ and -- are recognized before single + and -, removing ambiguity.

3. Taken Count in C Program:-

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    if (a > b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```
}
```

Tokens:-

Line

Tokens.

int main()

int main() = 4

{

1

int a = 10;

int a = 10; = 5.

int b = 20;

int b = 20; = 5.

Comment

0

if (a > b)

if (a > b) = 6.

return a;

return a; = 3

else

1

return b;

return b; = 3

}

1

Total Tokens = 26.

(Comments and whitespace are ignored.)

4. Lexical Analyzer performance:-

=> Processing Speed = 1,000,000 Characters/sec

=> Average token length = 8 Characters.

=> DFA transitions = 1.5 Per Character.

=> Token types = 20.

Step 1: Total Tokens:-

$$\frac{1,000,000}{8} = 125,000$$

Tokens Generated = 125,000

Step 2: DFA State Transitions.

$$1,000,000 \times 1.5 = 1,500,000$$

Total DFA Transitions = 1,500,000

Interpretation.

=> 125,000 tokens generated.

=> 1.5 million DFA Transitions Performed.

=> The lexical analyzer is efficient because DFA processing is linear ($O(n)$).

5. Lexical Analysis of

int var 123.12345;

float var 456 = 123.45;

if (var 123 == var 456) { return; }

Categorized Tokens:-

Keywords (4):-

- * int.
- * float
- * if.
- * return.

Identifiers (3)

=> var 123

=> var 456

=> var 123.

=> var 456 (appeared again; total identifier occurrences = 4).

Operators (2):-

=

==

Separators (7)

- * ;
- * ;
- * (
- *)
- * {
- * }
- * ;

Literals (2)

+ 12345

* 123.45

Total Tokens :-

Category.	Count.
Keywords.	4
Identifier occurrences.	4
operators.	2
separators.	7
Literals.	2.

Total Tokens = 19.
